

Dominando Python: De Estruturas de Dados a Funções

Material Didático Completo baseado nos conceitos e programas de capacitação da trilha Python da Digital Innovation One (DIO), sob autoria e curadoria de Guilherme Arthur de Carvalho.

1. Introdução

Este material didático foi elaborado para servir como um guia de referência aprofundado, cobrindo de forma estruturada e didática as principais estruturas de dados integradas do Python (Listas, Tuplas, Conjuntos e Dicionários) e o funcionamento detalhado de funções. O domínio dessas ferramentas é fundamental para o desenvolvimento de códigos limpos, performáticos e reutilizáveis (reaproveitamento de código) sob o paradigma de programação estruturada. Ao final, são apresentadas questões práticas com gabarito comentado para fixação dos conhecimentos.

2. Listas (classe list)

Listas em Python representam estruturas de dados que armazenam de maneira sequencial qualquer tipo de objeto. Elas são objetos **mutáveis**, o que significa que seus valores podem ser alterados após sua criação.

Criação de Listas

Podemos criar listas utilizando o construtor `list`, a função `range` para sequências numéricas, ou colocando valores separados por vírgula dentro de colchetes `[]`.

```
# Exemplos de criação de listas
frutas = ["laranja", "maca", "uva"]
lista_vazia = []
letras = list("python") # ["p", "y", "t", "h", "o", "n"]
numeros = list(range(5)) # [0, 1, 2, 3, 4]
carro = ["Ferrari", "F8", 4200000, 2020, "São Paulo", True] # Tipos mistos
```

Acesso Direto e Índices Negativos

O acesso aos dados é feito através de índices inteiros baseados em zero. Python também suporta indexação negativa, onde a contagem começa do final da lista a partir do índice `-1`.

```
frutas = ["maçã", "laranja", "uva", "pera"]
primeira = frutas[0] # "maçã"
terceira = frutas[2] # "uva"
ultima = frutas[-1] # "pera" (índice negativo)
penultima = frutas[-3] # "laranja" (conta de trás para frente)
```

Listas Aninhadas (Matrizes)

Como as listas podem armazenar qualquer objeto, podemos ter listas que armazenam outras listas, criando estruturas bidimensionais (tabelas e matrizes). O acesso é feito especificando os índices de linha e coluna.

```
matriz = [  
    [1, "a", 2],  
    ["b", 3, 4],  
    [6, 5, "c"]  
]  
linha_zero = matriz[0] # [1, "a", 2]  
elemento_especifico = matriz[0][0] # 1  
ultimo_elemento = matriz[-1][-1] # "c"
```

Fatiamento (Slicing)

Além do acesso direto, é possível extrair uma sub-sequência informando os parâmetros `[início:fim:passo]`. O cursor percorre a lista pulando de acordo com o passo determinado.

```
lista = ["p", "y", "t", "h", "o", "n"]  
sub_1 = lista[2:] # ["t", "h", "o", "n"] (do índice 2 até o fim)  
sub_2 = lista[:2] # ["p", "y"] (do início ao índice 2 exclusivo)  
sub_3 = lista[1:3] # ["y", "t"] (do índice 1 ao 3 exclusivo)  
sub_4 = lista[0:3:2] # ["p", "t"] (início ao 3, de 2 em 2)  
copia = lista[:] # ["p", "y", "t", "h", "o", "n"]  
invertida = lista[::-1] # ["n", "o", "h", "t", "y", "p"] (reverso)
```

Compreensão de Listas (List Comprehension)

A compreensão de listas oferece uma sintaxe curta e performática para criar novas listas com base em iteráveis existentes, aplicando filtros (estruturas condicionais) ou modificando os elementos.

```
numeros = [1, 30, 21, 2, 9, 65, 34]  
  
# Filtrando valores (pares) - Versão imperativa vs Compreensão  
pares_filtrados = [numero for numero in numeros if numero % 2 == 0]  
# Retorna: [30, 2, 34]  
  
# Modificando valores (potência quadrada)  
quadrados = [numero ** 2 for numero in numeros]  
# Retorna: [1, 900, 441, 4, 81, 4225, 1156]
```

Métodos Fundamentais da Classe list

A classe `list` possui diversos métodos internos de alta utilidade. A tabela abaixo detalha o funcionamento de cada um deles:

Método	Descrição	Exemplo de Uso
<code>append(obj)</code>	Adiciona um elemento ao final da lista.	<code>lista.append('Python')</code>
<code>clear()</code>	Remove todos os elementos da lista, deixando-a vazia.	<code>lista.clear()</code>
<code>copy()</code>	Retorna uma cópia rasa da lista.	<code>copia = lista.copy()</code>

Método	Descrição	Exemplo de Uso
<code>count(obj)</code>	Conta o número de vezes que um objeto aparece na lista.	<code>cores.count('vermelho')</code>
<code>extend(iter)</code>	Prolonga a lista adicionando elementos de um iterável.	<code>lista.extend(['java', 'js'])</code>
<code>index(obj)</code>	Retorna o índice da primeira ocorrência do objeto.	<code>lista.index('java')</code>
<code>pop([idx])</code>	Remove e retorna o elemento. Por padrão remove o último.	<code>lista.pop()</code> ou <code>lista.pop(0)</code>
<code>remove(obj)</code>	Remove a primeira ocorrência do elemento especificado.	<code>lista.remove('c')</code>
<code>reverse()</code>	Inverte a ordem dos elementos da lista in-place.	<code>lista.reverse()</code>
<code>sort([key, rev])</code>	Ordena os itens in-place. Aceita chaves de ordenação.	<code>lista.sort(key=lambda x: len(x))</code>
<code>len(lista)</code>	Função built-in que retorna o tamanho da lista.	<code>len(lista)</code> # retorna inteiro
<code>sorted(lista, ...)</code>	Função built-in que retorna uma nova lista ordenada.	<code>sorted(lista, reverse=True)</code>

IMPORTANTE

O método `sort()` modifica a lista original de forma definitiva (in-place), enquanto a função `sorted()` é uma função global que gera uma cópia ordenada, preservando a sequência original intacta.

3. Tuplas (classe tuple)

Tuplas são estruturas de dados muito parecidas com as listas. No entanto, a principal e mais importante diferença é que as **tuplas são imutáveis**, enquanto as listas são mutáveis. Uma vez criada, uma tupla não pode ter seus elementos alterados, adicionados ou removidos.

Criação de Tuplas

Podemos criar tuplas utilizando a classe `tuple` ou colocando valores separados por vírgulas dentro de parênteses `()`. Para declarar uma tupla com um único elemento, é obrigatório utilizar uma vírgula no final, caso contrário o Python interpretará apenas como uma expressão entre parênteses.

```
frutas = ("laranja", "pera", "uva",) # Boa prática: vírgula no final
letras = tuple("python") # ("p", "y", "t", "h", "o", "n")
numeros = tuple([1, 2, 3, 4]) # Conversão de lista para tupla
pais_singular = ("Brasil",) # Elemento único (requer vírgula!)
```

Acesso, Fatiamento e Tuplas Aninhadas

O acesso direto aos elementos por índices (positivos e negativos), fatiamento e aninhamento para estruturas bidimensionais funciona de maneira idêntica às listas. A única restrição é que qualquer tentativa de alteração resultará em erro (`TypeError`).

```
matriz_tupla = (  
    (1, "a", 2),  
    ("b", 3, 4),  
)  
elemento = matriz_tupla[0][-1] # 2  
# matriz_tupla[0][0] = 99 # Lança TypeError (imutável!)
```

Iteração e a Função `enumerate`

A iteração clássica é realizada com o laço `for`. Quando precisamos obter o índice do objeto que está sendo iterado dentro do laço, utilizamos a função built-in `enumerate()`.

```
carros = ("gol", "celta", "palio",)  
  
# Iterando sobre os itens  
for carro in carros:  
    print(carro)  
  
# Iterando com índice associado  
for indice, carro in enumerate(carros):  
    print(f"{indice}: {carro}")  
# Saída:  
# 0: gol  
# 1: celta  
# 2: palio
```

Métodos da Classe `tuple`

Devido à sua imutabilidade, a classe `tuple` possui um conjunto de métodos reduzido, limitando-se apenas a operações de busca e contagem que não alteram a estrutura:

```
cores = ("vermelho", "azul", "verde", "azul",)  
qtd_azul = cores.count("azul") # 2 (conta ocorrências)  
indice_verde = cores.index("verde") # 2 (busca a primeira posição)  
tamanho = len(cores) # 5 (função global len)
```

4. Conjuntos (classe `set`)

Um `set` é uma coleção não-ordenada de elementos únicos. Ele não permite elementos repetidos, sendo comumente utilizado para eliminar duplicidades de iteráveis e realizar operações matemáticas de conjuntos (união, interseção, diferença, etc.).

Criação de Conjuntos e Acesso aos Dados

Conjuntos podem ser criados passando um iterável para o construtor `set()` ou colocando valores entre chaves `{}`.

Atenção: conjuntos em Python **não suportam indexação ou fatiamento**. Caso queira acessar valores específicos por índice, é obrigatório converter o conjunto em uma lista.

```
# Eliminação automática de duplicatas na criação
numeros_set = set([1, 2, 3, 1, 3, 4]) # {1, 2, 3, 4}
letras_set = set("abacaxi") # {"b", "a", "c", "x", "i"}

# Acessando dados através de conversão
numeros = {1, 2, 3, 2}
numeros_lista = list(numeros)
primeiro_elemento = numeros_lista[0]
```

Iteração e Operações Matemáticas

Os conjuntos podem ser percorridos com o laço `for` e suportam a função `enumerate()`. Suas operações principais de álgebra de conjuntos e modificação de elementos incluem:

```
conjunto_a = {1, 2, 3}
conjunto_b = {2, 3, 4}

# União e Interseção
uniao = conjunto_a.union(conjunto_b) # {1, 2, 3, 4}
intersecao = conjunto_a.intersection(conjunto_b) # {2, 3}

# Modificação de Elementos
sorteio = {1, 23}
sorteio.copy() # Retorna cópia do conjunto {1, 23}
sorteio.discard(1) # Remove o item 1. Não lança erro caso não exista.
sorteio.discard(45) # Ignorado sem erros
sorteio.remove(23) # Remove o item 23. Lança KeyError caso não exista.
# sorteio.pop() # Remove e retorna um elemento arbitrário (aleatório)
```

Funções Úteis e Operadores

A função `len()` retorna a quantidade de elementos únicos existentes no conjunto. O operador `in` é extremamente otimizado em conjuntos para checar a existência de um elemento.

```
numeros = {1, 2, 3, 4, 5}
tamanho = len(numeros) # 5
existe_um = 1 in numeros # True
existe_dez = 10 in numeros # False
```

5. Dicionários (classe dict)

Um dicionário é um conjunto não-ordenado de pares do tipo `chave:valor`, onde as chaves são **únicas** em uma dada instância e devem obrigatoriamente corresponder a objetos imutáveis (como strings, números e tuplas).

Criação, Acesso e Modificação de Dados

Dicionários são delimitados por chaves `{}` contendo pares chave-valor separados por vírgula, ou podem ser instanciados usando o construtor `dict()`.

```
# Criação
pessoa = {"nome": "Guilherme", "idade": 28}
pessoa_alt = dict(nome="Guilherme", idade=28)

# Acesso direto e Modificação
pessoa["telefone"] = "3333-1234" # Adiciona nova chave-valor
nome_atual = pessoa["nome"] # "Guilherme"
pessoa["nome"] = "Maria" # Altera valor existente
```

Dicionários Aninhados

Dicionários podem armazenar qualquer objeto Python como valor, incluindo outros dicionários, permitindo criar estruturas de dados complexas e organizadas (como perfis de usuários).

```
contatos = {
    "guilherme@gmail.com": {"nome": "Guilherme", "telefone": "3333-2221"},
    "giovanna@gmail.com": {"nome": "Giovanna", "telefone": "3443-2121"}
}
telefone_giovanna = contatos["giovanna@gmail.com"]["telefone"] # "3443-2121"
```

Iteração em Dicionários

A forma mais comum de percorrer dicionários é através do laço `for`, que pode ser feito diretamente pelas chaves ou utilizando o método `.items()` para obter pares chave e valor simultaneamente.

```
# Iterando pelas chaves diretamente
for chave in contatos:
    print(chave, contatos[chave])

# Iterando com .items() (recomendado para pares chave-valor)
for chave, valor in contatos.items():
    print(chave, valor)
```

Métodos da Classe dict

Abaixo estão listados os métodos internos essenciais da classe `dict` para controle estrutural:

Método	Descrição	Exemplo Prático
<code>clear()</code>	Apaga todos os pares chave-valor do dicionário.	<code>contatos.clear() -> {}</code>
<code>copy()</code>	Retorna uma cópia rasa do dicionário.	<code>copia = contatos.copy()</code>
<code>fromkeys(keys, [v])</code>	Gera chaves com valores padrão (ou None se omitido).	<code>dict.fromkeys(['nome', 'idade'], 'vazio')</code>
<code>get(key, [default])</code>	Busca o valor da chave. Evita o erro <code>KeyError</code> se não existir.	<code>contatos.get('chave_inexistente', {})</code>
<code>items()</code>	Retorna uma lista de tuplas contendo pares chave-valor.	<code>contatos.items()</code>
<code>keys()</code>	Retorna uma visão com todas as chaves do dicionário.	<code>contatos.keys()</code>

Método	Descrição	Exemplo Prático
pop(key, [default])	Remove a chave especificada e retorna seu valor associado.	contatos.pop('guilherme@gmail.com')
popitem()	Remove e retorna o último par chave-valor inserido como tupla.	contatos.popitem() -> (chave, valor)
setdefault(key, d)	Insere a chave com o valor d se ela não existir. Retorna o valor.	contato.setdefault('idade', 28)
update(other_dict)	Atualiza o dicionário mesclando novos pares chave-valor.	contatos.update({'guilherme@gmail.com': {'nome': 'Gui'}})
values()	Retorna uma visão com todos os valores do dicionário.	contatos.values()

Palavras-chave in e del

A palavra-chave `in` é usada para verificar se uma determinada chave existe no dicionário. O comando `del` é utilizado para apagar chaves específicas, inclusive dentro de aninhamentos.

```
contatos = {
    "guilherme@gmail.com": {"nome": "Guilherme", "telefone": "3333-2221"},
    "giovanna@gmail.com": {"nome": "Giovanna", "telefone": "3443-2121"}
}

existe_gui = "guilherme@gmail.com" in contatos # True
existe_idade = "idade" in contatos["guilherme@gmail.com"] # False

del contatos["guilherme@gmail.com"]["telefone"] # Apaga telefone do Guilherme
del contatos["giovanna@gmail.com"] # Remove Giovanna por completo
```

6. Funções (def)

Uma função é um bloco de código identificado por um nome único que pode receber uma lista de parâmetros. Programar com base em funções faz parte da **programação estruturada**, tornando os códigos mais legíveis, modulares e permitindo o reuso eficiente de lógica.

Definição, Chamada e Parâmetros Padrão

Funções são declaradas com a palavra reservada `def`. Parâmetros podem ter valores padrões definidos diretamente na assinatura da função, tornando-os opcionais durante a invocação.

```
# Sem argumentos
def exibir_mensagem():
    print("Olá mundo!")

# Com argumento obrigatório
def exibir_mensagem_2(nome):
    print(f"Seja bem vindo {nome}!")

# Com argumento opcional (valor padrão)
def exibir_mensagem_3(nome="Anônimo"):
    print(f"Seja bem vindo {nome}!")

exibir_mensagem()
exibir_mensagem_2(nome="Guilherme") # Chamada nomeada
exibir_mensagem_3() # Utiliza o padrão "Anônimo"
```

Retorno de Valores

Para retornar um valor para quem chamou a função, utilizamos a palavra reservada **return**. Por padrão, se nenhum retorno for especificado, toda função Python retorna **None**. Python tem como grande diferencial a capacidade de retornar múltiplos valores simultaneamente (que são agrupados e entregues como uma tupla).

```
def calcular_total(numeros):
    return sum(numeros)

# Função retornando múltiplos valores (antecessor e sucessor)
def retorna_antecessor_e_sucessor(numero):
    antecessor = numero - 1
    sucessor = numero + 1
    return antecessor, sucessor

total = calcular_total([10, 20, 34]) # 64
ant, suc = retorna_antecessor_e_sucessor(10) # Desempacotamento de Tupla (9, 11)
```

Argumentos Nomeados (Dicionários como Kwargs)

Python permite mapear dicionários inteiros como argumentos nomeados utilizando a sintaxe de desempacotamento de dicionário de dois asteriscos (**).

```
def salvar_carro(marca, modelo, ano, placa):
    print(f"Carro inserido com sucesso! {marca}/{modelo}/{ano}/{placa}")

# Chamada clássica por posição ou nomeada
salvar_carro("Fiat", "Palio", 1999, "ABC-1234")

# Desempacotamento de dicionário
carro_dados = {"marca": "Fiat", "modelo": "Palio", "ano": 1999, "placa": "ABC-1234"}
salvar_carro(**carro_dados)
```

Args e Kwargs (*args e **kwargs)

Podemos programar funções com parâmetros dinâmicos de tamanho indeterminado utilizando ***args** e ****kwargs**. O primeiro recebe os valores posicionais excedentes em forma de uma **tupla**, e o segundo captura os valores nomeados em forma de um **dicionário**.


```
def exibir_poema(data_extenso, *args, **kwargs):
    texto = "\n".join(args)
    meta_dados = "\n".join([f"{chave.title()}: {valor}" for chave, valor in kwargs.items()])
    mensagem = f"{data_extenso}\n\n{texto}\n\n{meta_dados}"
    print(mensagem)

exibir_poema("Zen of Python", "Beautiful is better than ugly.", autor="Tim Peters", ano=1999)
```

Restrições de Parâmetros (Especiais)

Por padrão, argumentos podem ser passados de forma mista (posicional ou explicitamente pelo nome). No entanto, para melhor legibilidade e desempenho do interpretador, Python introduziu delimitadores especiais:

REGRAS DE SINTAXE

- **Barra (/):** Parâmetros antes da barra são restritos apenas à passagem por posição (Positional-only).
- **Asterisco (*):** Parâmetros após o asterisco são restritos apenas à passagem nomeada (Keyword-only).

```
# Exemplo de Parâmetros Híbridos (Positional-only / e Keyword-only *)
def criar_carro(modelo, ano, placa, /, *, marca, motor, combustivel):
    print(modelo, ano, placa, marca, motor, combustivel)

# Válido: posições antes de /, e nomes depois de *
criar_carro("Palio", 1999, "ABC-1234", marca="Fiat", motor="1.0", combustivel="Gasolina")

# Inválido: tentar passar modelo='Palio' por nome resulta em erro
# criar_carro(modelo="Palio", 1999, "ABC-1234", marca="Fiat", motor="1.0", combustivel="Gasolina")
```

Objetos de Primeira Classe

Em Python, funções são tratadas como objetos de primeira classe. Isso significa que podemos atribuir funções a variáveis, passá-las como parâmetros para outras funções, usá-las como elementos dentro de coleções (listas, dicionários) e retorná-las de outras funções (closures).

```
def somar(a, b):
    return a + b

# Recebendo uma função como parâmetro
def exibir_resultado(a, b, funcao):
    resultado = funcao(a, b)
    print(f"O resultado da operação {a} + {b} = {resultado}")

exibir_resultado(10, 10, somar) # Lança somar como objeto
```

Escopo Local e Escopo Global

Python trabalha com dois níveis de escopo: local (bloco interno da função) e global (fora de blocos). Variáveis locais em tipos imutáveis têm alterações descartadas quando a função termina. Para forçar a modificação de uma variável global dentro de uma função, utiliza-se a palavra-chave `global`. No entanto, **essa prática é altamente desencorajada** por quebrar a pureza estruturada do código.

```
salario = 2000

def salario_bonus(bonus):
    global salario # Informa que se refere à variável do topo
    salario += bonus
    return salario

salario_bonus(500) # Modifica salário global para 2500
```

7. Exercícios de Fixação

Abaixo estão propostas três questões abrangentes que integram todos os tópicos estudados neste material (listas, tuplas, conjuntos, dicionários e funções com parâmetros especiais). Tente resolver antes de ler as soluções detalhadas.

Questão 1 (Listas, Compreensão e Funções)

Enunciado: Escreva uma função em Python chamada `filtrar_e_quadradar` que receba uma lista de números inteiros como parâmetro por posição. A função deve aplicar uma *list comprehension* para filtrar apenas os números que são múltiplos de 3 e, em seguida, retornar uma nova lista contendo esses números elevados ao quadrado.

Código de Apoio (Gabarito):

```
def filtrar_e_quadradar(numeros, /): # / Restringe a posicional-only
    return [num ** 2 for num in numeros if num % 3 == 0]

# Teste prático:
dados = [1, 3, 5, 6, 9, 10, 12]
resultado = filtrar_e_quadradar(dados)
print(resultado) # Esperado: [9, 36, 81, 144]
```

Questão 2 (Tuplas, Conjuntos e Dicionários)

Enunciado: Um sistema de cadastro de eventos recebe uma tupla de participantes que confirmaram presença. Devido a múltiplos envios de formulário, a tupla contém nomes repetidos. Escreva uma função que receba essa tupla, utilize um conjunto (`set`) para eliminar nomes duplicados e retorne um dicionário onde cada chave seja o nome do participante único e o valor associado seja o número de caracteres de cada nome.

Código de Apoio (Gabarito):

```
def processar_participantes(participantes_tupla):
    # Elimina os duplicados ao converter para set
    participantes_unicos = set(participantes_tupla)

    # Cria o dicionário usando dicionário por compreensão
    return {nome: len(nome) for nome in participantes_unicos}

# Teste prático:
confirmados = ("Ana", "Bruno", "Ana", "Carlos", "Bruno", "Daniela")
registro = processar_participantes(confirmados)
print(registro)
# Saída esperada (chaves sem ordem específica):
# {'Carlos': 6, 'Daniela': 7, 'Ana': 3, 'Bruno': 5}
```

Questão 3 (Parâmetros Especiais e Escopo)

Enunciado: Analise a definição da função a seguir, explique quais restrições estão sendo impostas para os parâmetros `a`, `b`, `c` e `d` com base no uso de `/` e `*`, e indique qual será a saída do código quando executado:

```
acumulador = 100

def calcular_valores(a, b, /, *, c, d):
    global acumulador
    resultado = (a + b) * (c - d)
    acumulador += resultado
    return resultado

retorno = calcular_valores(10, 20, c=5, d=3)
print(f"Retorno: {retorno} | Acumulador Global: {acumulador}")
```

Resolução Detalhada:

1. Restrição de parâmetros:

- Os parâmetros **a** e **b** estão antes do delimitador **/**, o que significa que são **apenas posicionais** (Positional-only). Tentar chamá-los como **a=10** resultará em erro.
- Os parâmetros **c** e **d** estão após o delimitador *****, o que significa que são **apenas nomeados** (Keyword-only). Devem ser passados explicitamente como **c=valor** e **d=valor**.

2. Escopo e Acumulador:

- A instrução **global acumulador** diz que a função usará a variável do escopo global. O cálculo é **resultado = (10 + 20) * (5 - 3) = 30 * 2 = 60**. A variável global **acumulador** é atualizada para **100 + 60 = 160**.
- **Saída esperada:** Retorno: 60 | Acumulador Global: 160

Referências Bibliográficas

- CARVALHO, Guilherme Arthur de. *Python - Estruturas de Dados e Funções*. Digital Innovation One (DIO), 2024.
- DOCUMENTAÇÃO OFICIAL DO PYTHON. *Data Structures & Functions*. Disponível em python.org.